

TechniView D1: Detailed design

Header, scope, and conventions

Project title: TechniView Design Diagram

Goal statement: Create a web-based learning platform for technical computer science questions.

D1 scope: This document details the Database component and its grading and progress computations. It defers the Backend Server's remaining routes, the Code Runner internals, and the Web UI to D2. The design is based on repository commit 2257305, where the User Data DB and Question DB are table groups in one MySQL 8.0 database and the Backend Server calls Judge0.

Conventions: In D0, stick figures mean users, cylinders mean databases, squares mean user interfaces, and rectangles with inner lines mean servers. In Figure 1, each box is a table with only behavior-relevant fields. Dashed boxes are Question DB tables; solid boxes are User Data DB tables. Lines are relationships implemented with foreign keys; labels name the relationship. PK means primary key, FK means foreign key, UQ means membership in a unique constraint, and IX means indexed. A UQ or IX mark can belong to a composite constraint; the index table lists its exact columns. Crow's-foot marks show cardinality at both ends: a bar means one, a circle means zero, and a three-pronged foot means many.

Data model

D0 data model

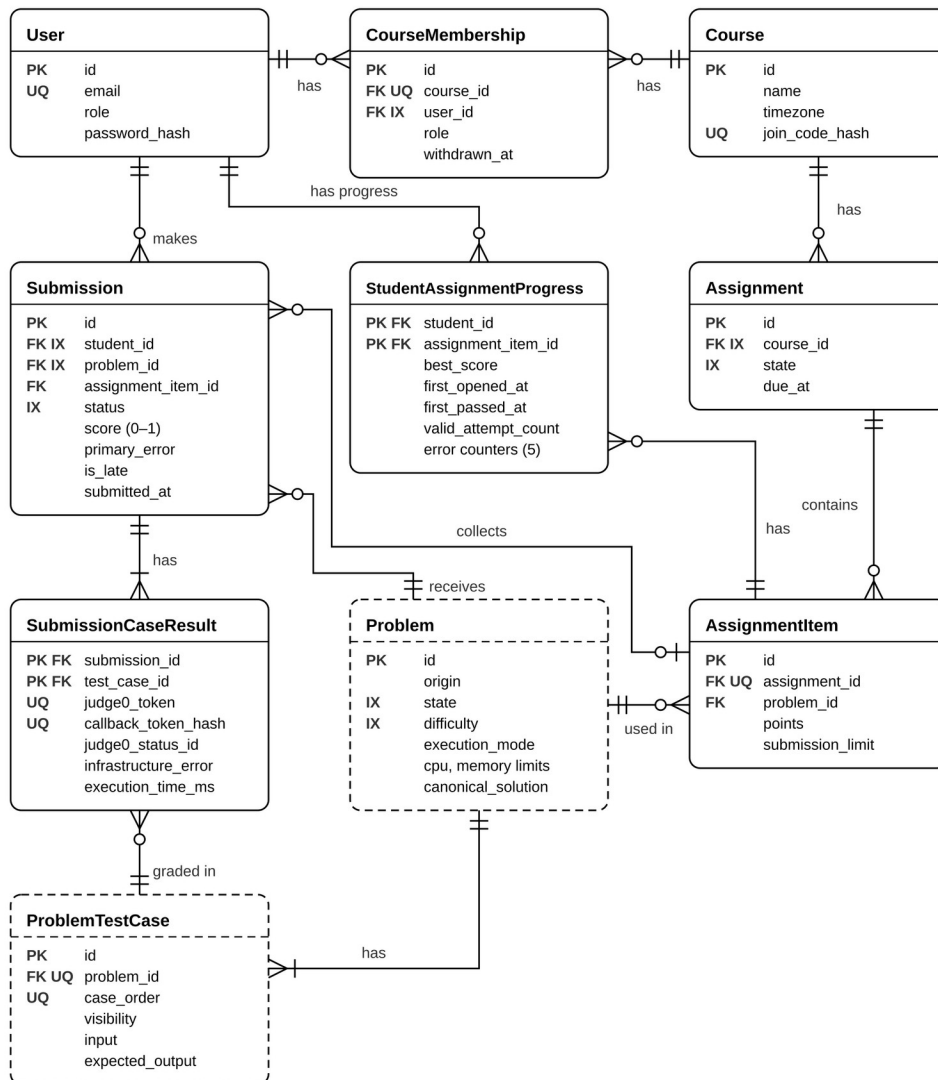
D0 showed two databases reached through the Backend Server: User Data DB for accounts, progress, and submissions (interface I9), and Question DB for prompts, difficulty, and test cases (interfaces I10, I11, and I13). D1 keeps these groups as tables in one MySQL database. Foreign keys protect row references; the submission service verifies that a selected assignment item belongs to the submitted problem before it inserts a submission.

D1 data model

Figure 1. Entity relationship diagram for the D1 data model.

Each submission fans out to one Judge0 result per test case

Dashed boxes: D0 Question DB tables. Solid boxes: D0 User Data DB tables. One MySQL 8.0 database.
PK primary key; FK foreign key; UQ unique constraint; IX indexed. Composite columns are listed below.



The diagram shows 10 tables used by grading and assignment progress, selected from the larger ORM model. It supports US-01 (submission and feedback), US-02 (student progress), and US-03 (assignments and assessment). `AssignmentItem.item_order` and `StudentAssignmentProgress.retry_count` are omitted from the compact drawing. `item_order` sets question order. The current grader gives each test case equal weight. Practice progress is stored in `StudentPracticeProgress`, outside this assignment-focused figure.

We chose a relational model because grading relies on foreign keys, unique constraints, transactions, and row locks. The composite foreign key from submissions to assignment_items requires an assigned submission's `problem_id` to match the item's `problem_id`. Foreign keys also protect the submission and test-case references; the service selects cases from the

submitted problem. Row locks protect callback and progress updates. A document store would move more of these checks into application code.

Structural decisions

Decision	Choice	Reason
Course roles	CourseMembership entity, not a role attribute on User	One person can be a TA in one course and a student in another; withdrawn_at keeps history. Requirement: US-03.
Problems in an assignment	AssignmentItem entity, not a plain link table	Carries points and submission_limit, and is copied from a question set so later edits cannot change a published assignment. Requirement: US-03.
Per-test results	SubmissionCaseResult entity, not a JSON column	Judge0 reports each case separately; each row holds its own token so one callback updates exactly one row. Requirement: US-01, AC-01.1.
Progress	StudentAssignmentProgress rows, not computed on each read	Updated once per finished submission, so dashboards read one row per student and item. Requirement: US-02, US-03.
Submission to assignment item	Zero-or-one to many, not many-to-many	A submission answers at most one assignment item; practice submissions leave it null. Requirement: US-01, US-03.
Difficulty and visibility	Attributes (enums), not entities	Fixed vocabularies with no data of their own. Requirement: US-01.

Index decisions

Indexed fields	Why
users(email) unique	Login lookup by email.
course_memberships(course_id, user_id) unique; course_id and user_id indexes	Unique membership lookup, course roster lookup, and reverse lookup by user.
problem_test_cases(problem_id, case_order) unique	Load one problem's cases in stable execution order.
assignments(course_id) and assignments(state)	Filter assignments by course and state.
problems(state) and problems(difficulty)	Filter and browse problems by publication state and difficulty.
submissions(student_id, problem_id)	Filter a student's submissions by problem.
submissions(status)	Filter queued or running submissions.
assignment_items(assignment_id, problem_id) unique	Prevent duplicate problems in one assignment.
assignment_items(assignment_id, item_order) unique	Prevent duplicate positions in an assignment.

Indexed fields	Why
assignment_items(id, problem_id) unique	Support the assignment-item/problem pair checked when a submission is created.
student_assignment_progress(student_id, assignment_item_id) primary key	Find or lock a student's assignment progress row.
submissions(id, problem_id) unique	Provide a unique submission/problem pair for related records.
submission_case_results(judge0_token) unique	Match a Judge0 report to its case row.
submission_case_results(callback_token_hash) unique	Find and authenticate callback URLs without storing the secret.
student_practice_progress(student_id, problem_id) primary key	Find or lock a student's practice progress row.
user_sessions(expires_at)	Find expired sessions during cleanup.

Core algorithms

Planning size from backend/data/problem_catalog.json: 500 problems and 8,371 test cases, about 16.7 cases per problem on average, median 10, maximum 122; 50 students and 30 assignment items per course; peak estimate 20 submissions per minute. The request rate is an estimate, not a measurement. US-01 and acceptance criterion AC-01.1 drive grading; US-02 and US-03 drive progress reporting.

Algorithm 1: submission grading

Purpose: Run a student's Python code against every test case in Judge0, then turn the reports into one status and score. A wrong verdict produces a wrong grade (US-01, AC-01.1).

Inputs and outputs: problem_id is a positive integer; assignment_item_id is an optional positive integer, null for practice; source_code is a UTF-8 string from 1 through 65,535 bytes. The server loads the ordered list of ProblemTestCase rows and CPU and memory limits (CPU seconds, default 2.0; memory, default 128,000 KB). `wait: bool` is an optional query parameter, default false. The submission output has status (enum: queued, running, passed, failed, compile_error, runtime_error, timeout, resource_limit, infrastructure_error), score (decimal from 0 to 1 or null until finalization), primary_error (enum or null), and one case-result row per test case.

Steps: Check that the problem is visible and, if assigned, that the item is published, available, and assigned to the caller. Enforce submission_limit, set is_late, create or lock the progress row, and insert a queued submission. Insert one result row per case with a random callback token stored as a SHA-256 hash. Then send one HTTP request per case to Judge0 in sequence, with limits, expected output, and callback URL. The optional wait query parameter defaults to false; wait=true makes each Judge0 call wait before the handler moves to the next case. Call-based problems use a wrapper that calls the student function and prints JSON. Each Judge0 job runs asynchronously on its workers. A callback locks its result row and stores the report. Finalization marks infrastructure errors separately; otherwise it calculates passed cases divided by total cases and selects the highest-priority failure in this order: compile error, timeout,

resource limit, runtime error, wrong answer.

Complexity and scale: For T test cases, the API makes T sequential Judge0 submission calls and writes $O(T)$ case-result rows; grading work is $O(T)$ plus Judge0 execution. At the planning peak, the catalog average means about 335 Judge0 jobs per minute; a 122-case problem can produce 2,440 jobs per minute if every submission uses that problem. At 100 times the estimated submission rate, the average becomes about 33,500 jobs per minute. These volumes matter; we have not measured service capacity. Measure request latency and queue depth before deciding whether to submit cases concurrently, batch requests, or add workers.

Why this design: Judge0 handles the security-sensitive code sandbox, which the team does not have experience building. One job per case gives each case its own limits and verdict. Callbacks return results after the browser closes; GET /api/submissions/{id} is the polling fallback.

Edge cases: Empty source is rejected with 422; a problem with no test cases returns 409. Duplicate callbacks and callbacks received after finalization do not change the grade. Failure ties use the fixed priority above. Judge0 failures return 429, 502, or 504; infrastructure failures do not count against the student's attempt limit.

Algorithm 2: progress aggregation

Purpose: For an assignment submission, update the student's StudentAssignmentProgress row, then summarize assignment progress for professors (D0 interface I6; US-02 and US-03).

Practice submissions update StudentPracticeProgress through the same finalization path; professor course analytics read assignment progress only. Incorrect counts mislead instructors.

Inputs and outputs: Update input is a finalized Submission row and its matching locked StudentAssignmentProgress row, or StudentPracticeProgress row when assignment_item_id is null. Output fields are best_score (decimal 0 to 1), first_opened_at and first_passed_at (UTC timestamps or null), valid_attempt_count (integer), and wrong_answer_count, compile_error_count, runtime_error_count, timeout_count, and resource_limit_count (integers). Read inputs are course_id (integer), optional assignment_id (integer), optional difficulty (easy, medium, or hard), optional tag (string), and limit/offset (integers). The response has assigned_count, started_count, completed_count, total_valid_attempts, total_retries, and five error counts (integers); completion_rate (ratio from 0 to 1 or null); average and median completion time (seconds or null); and a problem list with the same metrics.

Steps: On finalization, skip infrastructure errors, increment the valid attempt and matching error counter, and update best_score. Set first_passed_at on the first accepted submission that is on time or has been approved as late. On read, join assignment items in non-draft assignments to course recipients and membership rows, then left-join progress. Include archived assignments. Filter difficulty and tag, group records by problem in Python, sort by title and ID, and slice for limit and offset.

Complexity and scale: A progress update is $O(1)$ after the relevant row is found. A dashboard read is $O(R + P \log P)$, where R is assignment items $\times$ recipients and P is distinct problems. At the planning size, R is about 1,500; at 100 times that size, it is 150,000. Pagination slices after reading and sorting all records. The larger read may take seconds with grouping in Python; that

is a projection, not a measurement. The ORM model has course analytics rollup tables available if profiling shows this query is too slow. Pagination slices after loading and sorting all records.

Why this design: Write-time counters avoid recounting every submission for each dashboard read. `best_score` preserves a student's strongest result when they retry.

Edge cases: With no assigned rows, counts are zero and rates and completion times are null. Repeated callbacks do not double-count. Equal best scores leave the existing score unchanged. Draft assignments are excluded and archived assignments are included. Withdrawn students are currently included because the analytics query does not filter them; confirm that roster rule before D2.

Build versus reuse decisions

Piece	Build or Reuse	Library or Service	License	Maturity, Performance, Fit	Reason
Code sandbox	Reuse	Judge0	GPL-3.0	Established project; worker based; enforces CPU and memory limits; requires privileged containers.	The team has REST integration experience, not sandbox security experience. Run it as a separate service and review GPL obligations before redistribution.
Database	Reuse	MySQL 8.0	GPL-2.0	Mature server; row locks and composite keys fit the integrity needs; ample capacity for the estimated load.	Run the unmodified server as a separate container.
ORM and migrations	Reuse	SQLAlchemy 2, Alembic, PyMySQL	MIT	Established Python tools; synchronous transactions and indexed queries fit the current load.	The codebase uses typed SQLAlchemy models and Alembic revisions.
API framework	Reuse	FastAPI, Uvicorn, Pydantic	MIT, BSD-3-Clause	Established Python stack; schema validation and generated API docs fit the contract.	Reuse to validate request ranges and document JSON responses.

Piece	Build or Reuse	Library or Service	License	Maturity, Performance, Fit	Reason
Password hashing and dates	Reuse	Python hashlib.scrypt and datetime	PSF	Standard library; avoids another dependency for hashing and UTC date handling.	Do not hand-roll hashing or date parsing.
Sessions	Build	UserSession model, Python secrets and SHA-256	MIT	Small implementation; hashed, expiring, revocable server-side sessions.	Fits one API service; protect cookie with Secure, HttpOnly, and SameSite settings.
Grading and progress rules	Build	TechniView code	MIT	Small data volume; the domain rules are specific to assignment limits and scoring.	These rules are the product behavior described in the algorithms.
Problem bank	Reuse	backend/data/problem_catalog.json	MIT	500 audited problems; import validates required data and source version.	Retain APPS/APPS+ attribution and MIT license recorded in the catalog.

API contract

The detailed endpoints use JSON under /api and the techniview_session cookie. Successful requests return HTTP 200. ApiError and request-validation failures use {"error": {"code":string,"message":string,"details":object}}. Submission source_code is 1 to 65,535 UTF-8 bytes. Query limit defaults to 50 and must be 1–100; offset defaults to 0 and must be nonnegative.

Endpoint contract

Endpoint	Inputs (type, required, range)	Output
POST /api/submissions	JSON body: problem_id integer > 0, required; assignment_item_id integer > 0 or null, optional; source_code string, required, 1–65,535 UTF-8 bytes. Extra fields rejected. Optional wait query boolean, default false.	HTTP 200; Submission response defined below. With wait=false (default), it is usually queued or running; wait=true waits for each Judge0 job before responding.

Endpoint	Inputs (type, required, range)	Output
GET /api/submissions/{id}	Path id: integer, required. Existing submission IDs are positive; other values return 404. Cookie session required.	HTTP 200; current Submission response, visible only to its owner.
GET /api/courses/{course_id}/analytics/overview	Path course_id: integer. Optional query assignment_id integer; difficulty enum easy medium hard; tag string; limit integer 1–100 (default 50); offset integer >=0 (default 0). Staff membership required.	HTTP 200; {summary: Metrics, items: ProblemMetrics[], total: integer, limit: integer, offset: integer}. Metrics fields and units are specified below.

Submission response: id (integer); status (enum listed under Algorithm 1); done (boolean); score (decimal 0–1 or null); is_late and late_approved (booleans); passed_count and test_count (integers, including hidden cases); public_results (array of {case_order: integer, status_id: integer 1–14 or null, friendly_message: string}); poll_url (string path).

Analytics Metrics object: assigned_count, started_count, completed_count, total_valid_attempts, total_retries are integers; completion_rate is a ratio from 0 to 1 or null; avg_completion_seconds and median_completion_seconds are seconds or null; avg_retries_per_student is a number or null; errors is {wrong_answer, compile_error, runtime_error, timeout, resource_limit}, each an integer. Each items[] entry is {problem: ProblemSummary, metrics: Metrics}. ProblemSummary fields follow the problem summary response: id integer, title string, difficulty, origin, state enums, tags string[], typed_tags array, source_category string or null.

ApiError and request-validation failures use {"error":

{"code":string,"message":string,"details":object}}; validation details include field diagnostics.

Runner errors use FastAPI's {"detail":string} body. A runner failure marks the reserved submission infrastructure_error, which does not count toward the attempt limit.

Error responses

Status	Code	When it happens
401	authentication_required	Missing or expired session cookie.
403	staff_required	A student or non-staff member calls analytics.
404	course_not_found / problem_not_found / assignment_item_not_found / submission_not_found	Course or resource is missing, unavailable to caller, or not owned by caller.
409	problem_has_no_tests / submission_limit_reached	Problem has no cases or the assignment attempt limit is reached.
422	validation_error	Missing field, invalid type or range, extra field, or source exceeds byte limit.

Status	Code	When it happens
429	{detail: "runner busy, try again"}	Judge0 rejects the request because its service is busy.
502	{detail: "runner unavailable"}	Judge0 is unavailable or returns an invalid response.
504	{detail: "runner timed out"}	Judge0 does not respond before the request timeout.

```
{"problem_id": 1, "assignment_item_id": null, "source_code": "def sum_list(values):\n return sum(values)"}
```

```
POST /api/submissions
{"problem_id": 1, "source_code": "def sum_list(values):\n    return sum(values)\n"}
{"id": 42, "status": "queued", "done": false, "score": null, "is_late": false, "late_approved": false,
200 OK
{"id": 42, "status": "queued", "done": false, "score": null, "is_late": false,
"passed_count": 0, "test_count": 2,
"public_results": [{"case_order": 1, "status_id": 1,
"friendly_message": "In queue. Waiting for a worker."}],
"poll_url": "/api/submissions/42"}
```

Versioning: API routes will be placed under /api/v1 before external clients depend on them, and database changes use Alembic revisions. Removing or renaming a response field, changing a field's type or unit, or making an optional input required is a breaking API change.

Technology choices

Database: MySQL 8.0. Team skill fit: the team used MySQL in its databases course. License: GPL-2.0 for the server distribution. Community: MySQL has a large user and operator base. Performance: row locks, composite keys, and indexed lookups fit the expected course workload; the planned data volume is small for MySQL. Cost and hosting: the software is free to run in one container on the project VM. We chose MySQL over PostgreSQL because Judge0 uses PostgreSQL for its own data, and keeping TechniView's data in a separate database service isolates it from Judge0 resets.

Backend: Python 3.14 with FastAPI. Team skill fit: Python is the team's shared language and the language students submit. License: Python uses the PSF license; FastAPI uses MIT. Community: both have active ecosystems, and FastAPI generates API documentation from schemas. Performance: API work is small compared with code execution in Judge0. Cost and hosting: both are free to run in one container. We chose this over Node.js with Express so the backend and grading wrappers use the same language.

Front end: React, TypeScript, and Vite. Team skill fit: team members have used React in prior projects. License: React and Vite use MIT; TypeScript uses Apache-2.0. Community: all three have broad adoption and documentation. Performance: Vite builds static assets, and the expected course pages need no server-side rendering. Cost and hosting: the assets can be served from the same VM without a separate hosting service. The team considered a server-rendered Python front end, but has more prior experience with React.

Code execution and messaging: the repository's internal Judge0 fork (upstream version 1.13.1) with HTTP callbacks, Redis, and PostgreSQL for Judge0's own queue and state. Team skill fit: the team has experience calling REST services, but not building a secure sandbox. License: Judge0 is GPL-3.0 and runs as a separate service; review the license before modifying or redistributing it. Community: Judge0 is an established open-source judge. Performance: worker concurrency and per-case CPU and memory limits fit grading, but capacity must be measured at peak load. Cost and hosting: the software is free, though workers require privileged containers and may need a separate VM as load grows. The application sends one HTTP submission request per case and receives callbacks; Redis remains an internal Judge0 dependency, not a TechniView job queue.

Hosting: Docker Compose on one Linux VM. Team skill fit: the team uses Compose to start the same services locally and on the server. License: Docker Compose is Apache-2.0; the VM's operating system has its own distribution license. Community: Compose is widely used for small multi-service deployments. Performance: one VM is adequate for one or two courses under the stated estimates; move Judge0 workers to another VM if measurement shows contention. Cost and hosting: one university or student-credit VM keeps operating cost low. Kubernetes was considered but would add setup and operating work that this project's expected scale does not justify.